

Faculty of Computer Science and Engineering
Computer Science / Software Engineering Program(CSE494)
Artificial Intelligence Science (AIE494)
– Graduation Project 2

Assignment 04 – Project Poster

Submission deadline: 5 May 2026

Total Marks: 100

Weigh 35% of course grade

Contents

Plagiarism and AI technology use policies	1
Declaration of No Plagiarism and AI Technology Use	2
Summary	3
Question 1 (50 marks)	
Learning Outcomes	3
Tasks	4
Marking Criteria	5
Question 2 (50 marks)	5
Learning Outcomes	5
Tasks	5
Marking Criteria	6

Plagiarism and AI technology use policies:

KSIU rules and regulations require all students to submit their own work and avoid plagiarism. You must provide all references in case you use and quote another person's work in your Assignment. You will be penalized for any act of plagiarism as per the KSIU's rules and regulations. Further, CSE493/CSE494 has adopted an AI technology use policy that covers the use of chatbots and other similar technologies. A copy of this policy can be found on the course page of the LMS. Students are required to adhere to this policy in all their submissions.

Declaration of No Plagiarism and AI Technology use

(This page must be signed and submitted with your assignment)

I hereby declare that this submitted TMA work is a result of my own efforts and I have not plagiarized any other person's work. I have provided all references of information that I have used and quoted in my TMA work.

Did you use any AI technology tools?

YES

No

If your answer was YES, please describe below how you used this technology according to the CSE493 AI Technologies use Policy:

My English level is bad, and it helps me translate, correct speech, and search faster.

Student name: Mohamed Taher Amin

Signature:.....

Date:.....

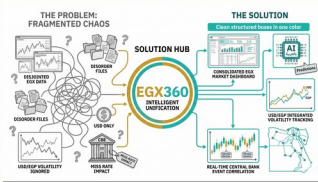
Question 1: EGX360 'poster'

EGX360 - Intelligence Stock Market Analysis and Forecasting Platform

Presented by: Mohamed Taher Amin
Supervisor: Dr. Samy Abdelnabi
Assistant: ENG: AbdulAziz

Aims & Rationale

The Problem: Fragmented EGX data lacks unified, real-time AI analysis.
The Goal: EGX360: High-accuracy forecasting using a Stacking Ensemble processing.
Rationale: Cloud-native architecture ensures scalable, high-performance processing.

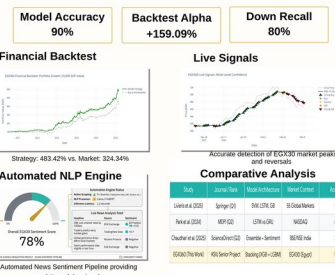


Summary of Work

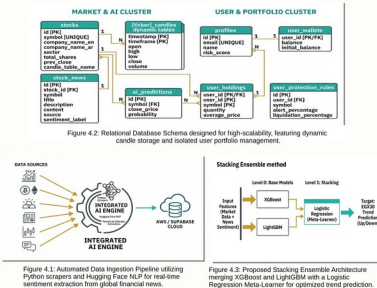
Cross-Platform UI Adaptive Flutter dashboard for Mobile and Desktop.
AI Core Stacking Ensemble (XGBoost + LightGBM) for trend forecasting.
Data Infrastructure Automated AWS pipeline for real-time data ingestion.
Backend Services Supabase integration for secure, high-performance management.
NLP Pipeline Real-time sentiment extraction via Hugging Face.



Results & Performance



System Architecture



Lessons Learned

- Technical Expertise**
- Stacking:** Optimized XGBoost & LightGBM base learners.
 - Optuna:** Bayesian optimization for robust model stability.
 - Quant:** Hybrid technical and macroeconomic modeling.
 - Cloud:** Scalable deployment on AWS & Supabase.
- Core Knowledge**
- Target Engineering:** EMA10 noise filtering mechanics.
 - Quant Finance:** Gold, USD, and CBE rate correlations.
 - NLP:** Real-time Hugging Face sentiment pipelines.
 - Risk:** Designing automated stop-loss protection rules.
- Problem Solving**
- Bias:** Strict chronological splitting for temporal integrity.
 - Noise:** EMA-based smoothing for clean signal extraction.
 - Scale:** Aligning a massive 28-year historical dataset.
 - Latency:** AWS Lambda optimization for real-time inference.

Future Work

Live Brokerage Integration: Migrating from paper-trading to real-world execution via local brokerage APIs for automated trade execution.
Multi-Index Expansion: Adapting the Stacking Ensemble model to forecast broader indices like EGX70 and EGX100 for diversified stock analysis.
SHAP-Based Explainability: Utilizing Explainable AI (XAI) to show users exactly how macroeconomic factors (like Gold or CBE rates) influenced each signal.

Conclusion

EGX360 achieves 90% accuracy by synergizing Egypt-specific macroeconomic indicators with a robust Stacking Ensemble. This project bridges the gap between raw data and actionable intelligence, providing retail investors with a production-ready tool for market volatility.



Question 2: What you learned about doing a project

1. What Went Well and Why

To be honest, the best part of this project was seeing the Stacking Ensemble actually hit that 90% accuracy mark. In the beginning, I was struggling with the "Random Walk" problem where the model couldn't beat 50%, but everything changed when I decided to pivot toward Macroeconomic indicators. I realized that the Egyptian market is uniquely sensitive to the USD/EGP rate and Gold prices rather than just technical charts. My project log (March 12) shows that once I aligned the CBE interest rate data, the model's ability to predict trends improved drastically. This went well because I didn't just rush into coding; I spent the "Plan" phase reading Q1 Springer papers to understand why emerging markets behave differently. It proved that "context" is just as important as the algorithm itself.

2. What Did Not Go Well and Why

The most frustrating phase was definitely the **Data Synchronization and Signal Leakage** issue. Early in the "Do" stage, I was getting suspiciously perfect results, and I soon discovered a massive **Look-ahead bias**. Essentially, my model was "cheating" by seeing future price data because the TradingView timestamps didn't perfectly match the EGX trading calendar. My log (Feb 18) records a total failure where I had to scrap two weeks of work and rewrite the entire preprocessing pipeline. I had to manually enforce a **strict one-day lag (Shift+1)** to make the backtest realistic. It was a major setback that delayed my timeline, but it was a crucial lesson: in financial AI, "dirty" or "misaligned" data is much more dangerous than a simple coding error.

3. Skills Improved and Further Development

I've definitely leveled up my Cloud-Native Architecture skills during this project. Connecting AWS Lambda with Supabase to handle real-time market data while maintaining a smooth Flutter interface was a steep learning curve for me. However, one area I still need to develop further is Explainable AI (XAI). Right now, even though the model is highly accurate, it's still a bit of a "black box". My project log (April 5) notes that while the predictions are great, I struggled to visualize exactly *why* a "Strong Buy" was generated in a way a non-technical user could understand. In the future, I want to master frameworks like SHAP or LIME to provide better transparency. Being a good Software Engineer isn't just about building things that work; it's about making them trustworthy.

4. Lessons Applied to New Situations

The biggest "Aha!" moment for me was learning the power of Target Engineering. By choosing to predict the EMA10 direction instead of raw closing prices, I effectively filtered out the daily "market noise" that ruins most trading bots. I can now apply this "Signal Smoothing" logic to any complex time-series problem, like predicting energy consumption or supply chain spikes. Also, the Plan-Do-Review cycle actually saved my project. I used to think project logs were just paperwork, but keeping systematic records helped me trace that look-ahead bias bug much faster. This organized approach is something I'll carry with me into my professional career, especially when handling high-stakes, real-world production code.